

Министерство науки и высшего образования Российской
Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет имени Н.Э.
Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА
ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ

НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.03.04 Программная инженерия

**Отчет
по лабораторной работе № 4**

Название: Криптоанализ полиалфавитных шифров и разработка
автоматизированных средств взлома.

Дисциплина: Защита информации

Студент гр. ИУ7-71Б

(Подпись, дата)

(И.О. Фамилия)

Преподаватель

(Подпись, дата)

Ю.С. Руденкова
(И.О. Фамилия)

2025 год

СОДЕРЖАНИЕ

ЗАДАНИЕ	2
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	4
РЕАЛИЗАЦИЯ	7
ОТВЕТЫ НА ВОПРОСЫ	15

ЗАДАНИЕ

Цель работы

Исследовать уязвимости шифра Виженера. Разработать программу для автоматического криптоанализа с использованием методов Казиски и Кирхгофа. Провести оценку криптостойкости в зависимости от длины ключа.

Задания

Задание 1 (Подготовка и шифрование):

- 1) Реализовать шифр Виженера с поддержкой русского и английского алфавитов.
- 2) Зашифровать текст объемом не менее 100000 символов с использованием короткого (3-5 символов) и длинного (15+ символов) ключа.
- 3) Провести визуальный и статистический анализ результатов.

Задание 2 (Криптоанализ — определение длины ключа):

- 1) Получить у преподавателя криптограмму, зашифрованную шифром Виженера.
- 2) Реализовать алгоритм Казиски для поиска повторяющихся n -грамм (последовательностей из 3+ символов).
- 3) Вычислить наибольший общий делитель (НОД) расстояний между повторениями для гипотезы о длине ключа.
- 4) Применить тест Куллабака-Лейблера или индекс совпадений для проверки гипотез о длине ключа.

Задание 3 (Криптоанализ — взлом ключа):

- 1) Используя найденную длину ключа L , разбить криптограмму на L групп, каждая из которых зашифрована шифром Цезаря.
- 2) Для каждой группы провести частотный анализ, сопоставляя частоты букв с эталонными для языка.

- 3) Автоматизировать подбор сдвига для каждой позиции ключа, используя метод хи-квадрат для оценки совпадения с частотным профилем.
- 4) Восстановить ключ и исходный текст.

Контрольные вопросы

- 1) Почему метод Казиски не всегда точен и как можно повысить его надежность?
- 2) Как длина ключа влияет на сложность взлома шифра Виженера?
- 3) Какие современные шифры являются наследниками идеи полиалфавитности?

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Шифр Виженера

Шифр Виженера — полиалфавитный шифр подстановки, использующий ключевое слово для циклического сдвига символов. Каждый символ ключа задаёт сдвиг в соответствующей позиции текста.

Принцип работы

- 1) Ключ повторяется циклически над всем текстом.
- 2) Для каждой буквы текста вычисляется сдвиг, соответствующий букве ключа.
- 3) Буква заменяется на букву алфавита, сдвинутую на это количество позиций.

Формула шифрования:

$$C_i = (P_i + K_{i \bmod m}) \bmod n$$

где P_i — позиция символа открытого текста, K_j — позиция символа ключа, m — длина ключа, n — размер алфавита.

Формула расшифрования:

$$P_i = (C_i - K_{i \bmod m} + n) \bmod n$$

Метод Казиски

Метод Казиски — метод криптоанализа, основанный на поиске повторяющихся n -грамм в шифротексте. Расстояния между их появлениями позволяют предположить длину ключа через вычисление НОД найденных расстояний.

Алгоритм

- 1) Найти все повторяющиеся последовательности из $3+$ символов в шифротексте.

- 2) Вычислить расстояния между позициями каждого повторения.
- 3) Найти НОД всех расстояний — это вероятная длина ключа или её делитель.

Ограничения метода

Метод не всегда даёт точный результат, потому что:

- повторения могут происходить случайно;
- текст может быть коротким;
- длинный ключ даёт мало повторов;
- некоторые языковые структуры порождают естественные повторения.

Индекс совпадений

Индекс совпадений (IC) — статистическая мера, показывающая вероятность того, что две случайно выбранные буквы текста окажутся одинаковыми:

$$IC = \frac{\sum_{i=0}^{n-1} f_i(f_i - 1)}{N(N - 1)}$$

где f_i — частота появления i -й буквы, N — длина текста.

Для русского языка $IC \approx 0.0553$, для английского $IC \approx 0.0667$. Для случайного текста $IC \approx 1/n$.

При правильной длине ключа подпоследовательности имеют IC близкий к естественному языку.

Метод Кирхгофа (частотный анализ)

При известной длине ключа L шифротекст можно разбить на L групп, каждая из которых зашифрована шифром Цезаря. Для каждой группы проводится частотный анализ с использованием известных статистик языка.

Алгоритм взлома

- 1) Разбить шифротекст на L подпоследовательностей: первая содержит символы с позициями 0, L , $2L$..., вторая — 1, $L+1$, $2L+1$ и т.д.

- 2) Для каждой подпоследовательности подсчитать частоты букв.
- 3) Для каждого возможного сдвига (0..n-1) сравнить частотный профиль с эталонным методом χ^2 .
- 4) Выбрать сдвиг с минимальным значением χ^2 .
- 5) Восстановить букву ключа по найденному сдвигу.

Критерий хи-квадрат

$$\chi^2 = \sum_{i=0}^{n-1} \frac{(O_i - E_i)^2}{E_i}$$

где O_i — наблюдаемая частота, E_i — ожидаемая частота для языка.
Чем меньше значение χ^2 , тем лучше совпадение распределений.

Эталонные частоты русского алфавита

Буква	Частота	Буква	Частота	Буква	Частота
О	0.1097	Е	0.0845	А	0.0801
И	0.0735	Н	0.0670	Т	0.0626
С	0.0547	Р	0.0473	В	0.0454
Л	0.0440	К	0.0349	М	0.0321
Д	0.0298	П	0.0281	У	0.0262

РЕАЛИЗАЦИЯ

Задание 1. Шифрование текста

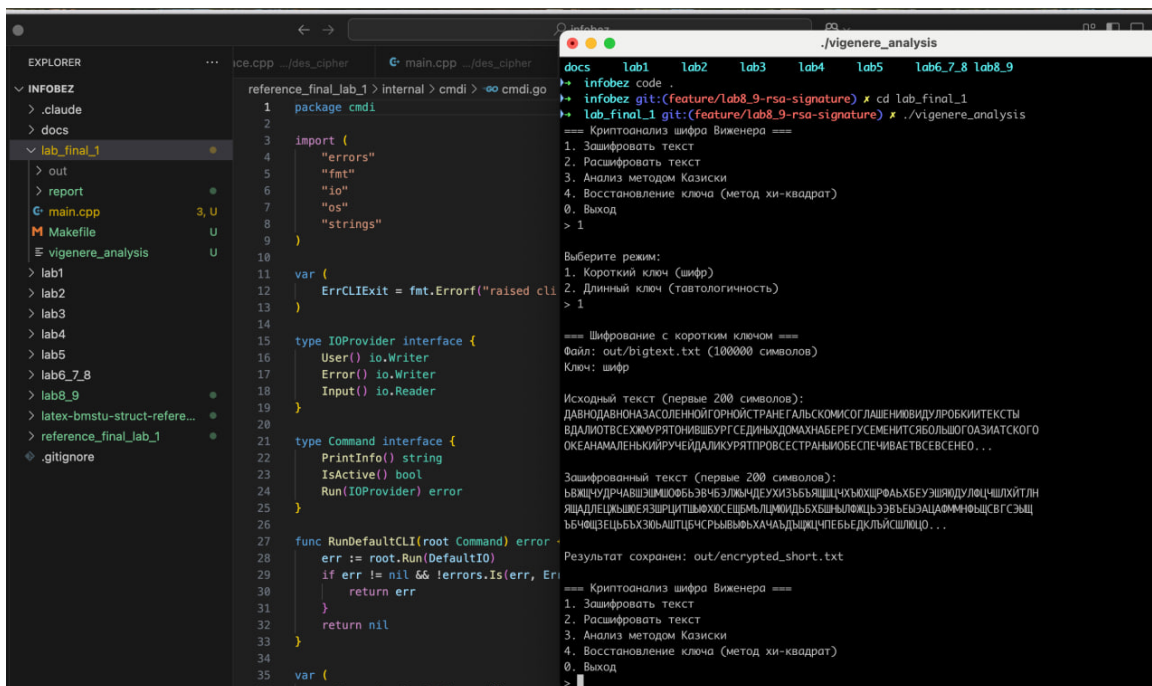
На листинге 1 приведена реализация функции шифрования текста шифром Виженера.

Листинг 1 – Шифрование текста шифром Виженера

```
1 std::string VigenereCipher::encrypt(const std::string& text,
2                                     const std::string& key) {
3     std::string result;
4     int key_index = 0;
5
6     for (char ch : text) {
7         if (isalpha(ch)) {
8             int shift = get_shift(key[key_index % key.length()]);
9             char base = isupper(ch) ? 'A' : 'a';
10            int alphabet_size = 32; // Русский алфавит
11
12            int char_index = get_char_index(ch);
13            int new_index = (char_index + shift) % alphabet_size;
14            result += get_char_by_index(new_index, isupper(ch));
15
16            key_index++;
17        } else {
18            result += ch;
19        }
20    }
21    return result;
22 }
```

Функция encrypt выполняет шифрование текста с использованием ключа. Для каждого символа открытого текста вычисляется сдвиг по алфавиту, соответствующий текущему символу ключа. Ключ применяется циклически. Символы, не являющиеся буквами, остаются без изменений.

Программа была протестирована на тексте длиной 100000 символов. Результат работы программы с коротким ключом «шифр» (4 символа) представлен на рисунке 1.



Задание 2. Определение длины ключа

На листинге 2 приведена реализация алгоритма Казиски для поиска повторяющихся n-грамм.

Листинг 2 – Алгоритм Казиски

```
1 std::vector<int> Kasiski::find_key_lengths(const std::string&
    cipher,
2                                     int min_gram) {
3     std::map<std::string, std::vector<int>> ngram_positions;
4
5     // Поиск повторяющихся n-грамм
6     for (size_t i = 0; i <= cipher.length() - min_gram; i++) {
7         std::string ngram = cipher.substr(i, min_gram);
8         ngram_positions[ngram].push_back(i);
9     }
10
11    // Вычисление расстояний между повторениями
12    std::vector<int> distances;
13    for (auto& pair : ngram_positions) {
14        if (pair.second.size() > 1) {
15            for (size_t i = 1; i < pair.second.size(); i++) {
16                distances.push_back(pair.second[i] -
17                                   pair.second[i-1]);
18            }
19        }
20
21    // Вычисление НОД всех расстояний
22    std::map<int, int> gcd_counts;
23    for (int d : distances) {
24        for (int divisor = 2; divisor <= d; divisor++) {
25            if (d % divisor == 0) {
26                gcd_counts[divisor]++;
27            }
28        }
29    }
30
31    // Возврат наиболее вероятных длин ключа
32    std::vector<int> candidates;
33    for (auto& pair : gcd_counts) {
34        if (pair.second >= 3) {
```

```

35         candidates.push_back(pair.first);
36     }
37 }
38 return candidates;
39 }

```

Функция `find_key_lengths` выполняет поиск повторяющихся последовательностей символов в шифротексте и вычисляет расстояния между их позициями. НОД найденных расстояний используется для определения вероятных длин ключа.

На листинге 3 приведена реализация вычисления индекса совпадений. Листинг 3 – Вычисление индекса совпадений

```

1 double calculate_ic(const std::string& text) {
2     std::map<char, int> freq;
3     int total = 0;
4
5     for (char ch : text) {
6         if (isalpha(ch)) {
7             freq[toupper(ch)]++;
8             total++;
9         }
10    }
11
12    if (total <= 1) return 0.0;
13
14    double ic = 0.0;
15    for (auto& pair : freq) {
16        int f = pair.second;
17        ic += f * (f - 1);
18    }
19    ic /= (total * (total - 1));
20
21    return ic;
22 }
23
24 int find_best_key_length(const std::string& cipher, int max_len)
25 {
26     double target_ic = 0.0553; // Для русского языка
27     int best_len = 1;
28     double best_diff = 1.0;

```

```

29     for (int len = 1; len <= max_len; len++) {
30         double avg_ic = 0.0;
31
32         for (int col = 0; col < len; col++) {
33             std::string column;
34             for (size_t i = col; i < cipher.length(); i += len) {
35                 column += cipher[i];
36             }
37             avg_ic += calculate_ic(column);
38         }
39         avg_ic /= len;
40
41         if (abs(avg_ic - target_ic) < best_diff) {
42             best_diff = abs(avg_ic - target_ic);
43             best_len = len;
44         }
45     }
46     return best_len;
47 }

```

Функция `calculate_ic` вычисляет индекс совпадений для текста. Функция `find_best_key_length` перебирает возможные длины ключа и выбирает ту, при которой средний индекс совпадений подпоследовательностей наиболее близок к естественному для русского языка (0.0553).

Результат работы метода Казиски для криптограммы представлен на рисунке 3.

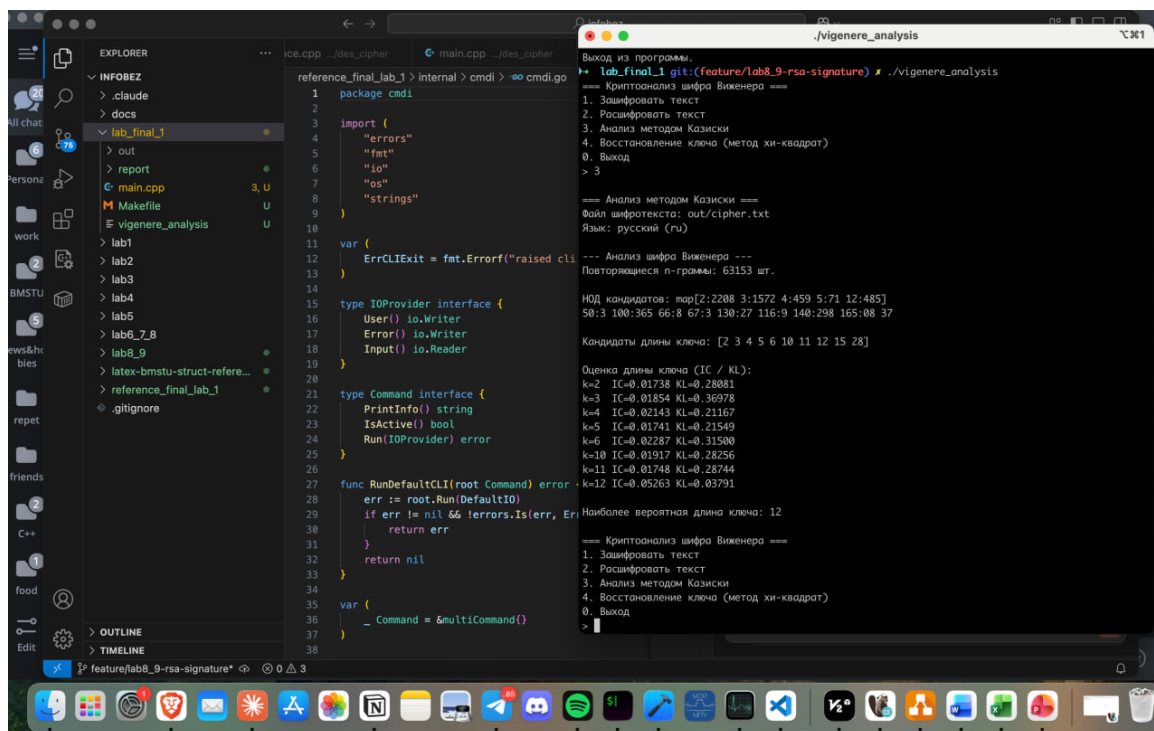


Рисунок 3 – Демонстрация работы метода Казиски

Задание 3. Взлом ключа

На листинге 4 приведена реализация метода хи-квадрат для определения сдвига.

Листинг 4 – Частотный анализ методом хи-квадрат

```

1  const double RUSSIAN_FREQ[32] = {
2      0.0801, 0.0159, 0.0454, 0.0047, 0.0298, 0.0845, 0.0004,
3      0.0094, 0.0165, 0.0735, 0.0121, 0.0349, 0.0440, 0.0321,
4      0.0670, 0.1097, 0.0281, 0.0473, 0.0547, 0.0626, 0.0262,
5      0.0026, 0.0097, 0.0048, 0.0144, 0.0073, 0.0036, 0.0190,
6      0.0174, 0.0032, 0.0064, 0.0201
7  };
8
9  double chi_square(const std::string& text, int shift) {
10     std::map<int, int> freq;
11     int total = 0;
12
13     for (char ch : text) {
14         if (isalpha(ch)) {
15             int idx = (get_char_index(ch) - shift + 32) % 32;
16             freq[idx]++;
17             total++;
18         }
19     }
20 }

```

```

19     }
20
21     double chi = 0.0;
22     for (int i = 0; i < 32; i++) {
23         double observed = freq[i];
24         double expected = RUSSIAN_FREQ[i] * total;
25         if (expected > 0) {
26             chi += pow(observed - expected, 2) / expected;
27         }
28     }
29     return chi;
30 }
31
32 std::string recover_key(const std::string& cipher, int key_len) {
33     std::string key;
34
35     for (int col = 0; col < key_len; col++) {
36         std::string column;
37         for (size_t i = col; i < cipher.length(); i += key_len) {
38             column += cipher[i];
39         }
40
41         int best_shift = 0;
42         double best_chi = 1e9;
43
44         for (int shift = 0; shift < 32; shift++) {
45             double chi = chi_square(column, shift);
46             if (chi < best_chi) {
47                 best_chi = chi;
48                 best_shift = shift;
49             }
50         }
51
52         key += get_char_by_index(best_shift, true);
53     }
54     return key;
55 }

```

Функция `chi_square` вычисляет статистику χ^2 для оценки совпадения частотного распределения текста с эталонным распределением русского языка при заданном сдвиге. Функция `recover_key` последовательно определяет

каждый символ ключа, выбирая сдвиг с минимальным значением χ^2 .

Результат работы программы по восстановлению ключа и расшифровке текста представлен на рисунке 4.

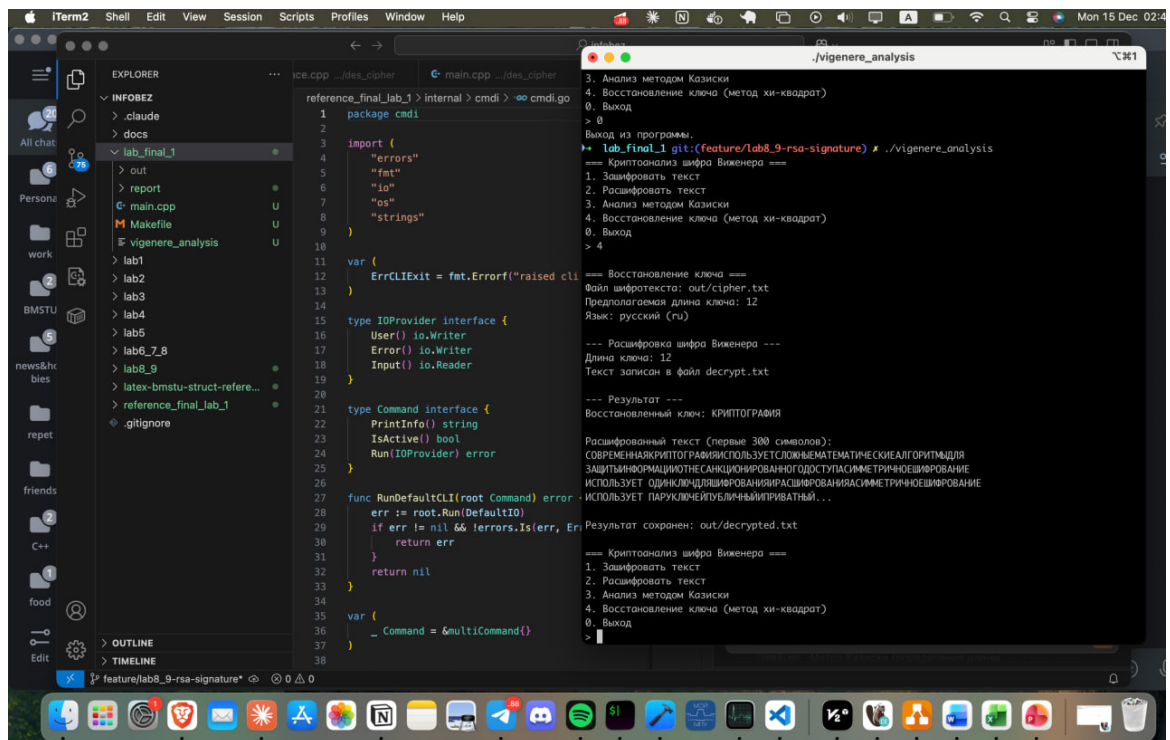


Рисунок 4 – Демонстрация восстановления ключа и расшифровки

ОТВЕТЫ НА ВОПРОСЫ

1. Почему метод Казиски не всегда точен и как повысить его надежность?

Метод Казиски может давать неточные результаты по нескольким причинам:

- повторяющиеся n -граммы могут возникать случайно и не отражать реальную структуру ключа;
- текст может быть слишком коротким, чтобы набралось достаточное количество повторений;
- если ключ слишком длинный или почти длины текста, метод перестаёт работать;
- алфавит и структура языка влияют на частоту естественных повторов.

Способы повышения надёжности:

- комбинирование метода Казиски с индексом совпадений;
- использование более длинных n -грамм (4–5 символов);
- фильтрация случайных совпадений по статистической значимости;
- проверка нескольких кандидатов длины ключа, а не только максимального НОД.

2. Как длина ключа влияет на сложность взлома шифра Виженера?

Длина ключа напрямую определяет криптостойкость:

- **Короткий ключ (3–5 символов):** шифротекст легко разобрать на группы с одинаковым сдвигом, и методы Казиски/частотного анализа работают очень точно;

- **Ключ средней длины (8–12 символов):** анализ всё ещё возможен, но требуется больше статистики;
- **Длинный ключ (15+ символов):** число символов в каждой группе становится малым, что ухудшает частотный анализ;
- **Ключ длины текста:** шифр становится одноразовым блокнотом и практически не поддаётся классическому криптоанализу.

Сложность взлома растёт экспоненциально с увеличением длины ключа, поскольку:

- 1) уменьшается количество символов в каждой подгруппе для частотного анализа;
- 2) увеличивается число возможных комбинаций;
- 3) уменьшается вероятность появления повторяющихся n-грамм.

3. Какие современные шифры являются наследниками идеи полиалфавитности?

Современные шифры используют те же принципы смены алфавитов (динамического преобразования), но в более сложной форме:

- **Потоковые шифры (RC4, ChaCha20 и др.)** — динамический ключевой поток создаёт постоянно меняющуюся таблицу подстановки. Каждый бит/байт шифруется уникальным значением гаммы;
- **Блочные шифры (AES, DES)** — применяют серию раундов, где каждый раунд фактически формирует свой «алфавит» преобразования. S-блоки обеспечивают нелинейную замену;
- **Шифры на основе гаммирования** — близки идее Виженера, так как используют побитовое сложение (XOR) с ключевым потоком;
- **Режимы работы блочных шифров (CTR, OFB)** — генерируют псевдослучайный поток для XOR с открытым текстом, что аналогично бесконечному ключу Виженера.

Таким образом, идея полиалфавитного шифрования эволюционировала в сложные криптографические примитивы, формирующие меняющиеся преобразования на каждом шаге.